Expected Gradient Divergence Weighting (EGDW) for Robust Memory Updates in Titans Language Models

Nishchay Jasuja
School of Computer Science
Carnegie Mellon University
njasuja@andrew.cmu.edu

Aditya Dewan
School of Computer Science
Carnegie Mellon University
adewan2@andrew.cmu.edu

Pranav Ram
School of Computer Science
Carnegie Mellon University
pram@andrew.cmu.edu

Abstract

Over the past 5 years, advancements in deep learning have been centered around transformer-based models, most notably Large Language Models. They achieve strong performance across the board but rely on self-attention, a mechanism that incurs an expensive quadratic cost in sequence length. Recent recurrent architectures, such as Mamba, offer linear inference time through compressed representations of historical data. However, these approaches see degraded performance on long-context tasks due to their inherent representation limits. Google's TITANS architecture aims to address this by mimicking the human memory system: short-term memory via standard attention, long-term memory via a novel neural memory module, and fixed memory via a set of persistent weights. The neural memory module is updated using a "surprise"-based rule, heavily dependent on gradient magnitude and thus susceptible to catastrophic memory updates when outlier gradients are encountered. We propose Expected Gradient Divergence Weighting (EGDW), a probabilistic update method for the neural memory module. EGDW models gradients as instances of a random variable and down-weighs gradients that are unlikely under the current gradient distribution, using local and global surprise terms derived from fundamental probabilistic bounds. On a 10B-token subset of FineWebEdu, EGDW-TITANS achieves lower validation cross-entropy than the original TITANS baseline with higher train loss, indicating greater generalization. This outlines the potential for novel probabilistic-divergence-based gradient descent algorithms to stabilize memory updates and improve long-context abilities in language models and neural networks at large.

1 Motivation and Research Objectives

1.1 Motivation

Modern Large Language Models (LLMs) based on the transformer architecture Vaswani et al. [2023] have proven to be highly effective for a wide range of natural language tasks, making these models

one of the most important advancements in deep learning in recent years. As such, through this project, we aimed to build on prior research efforts to develop architectural improvements for LLMs. Initially, we focused on the Tiny Recursive Model (TRM), a novel architecture introduced by Samsung in 2025. Whereas the transformer architecture applies the mechanism of self-attention, the TRM utilizes two hidden states and updates these hidden states in a recursive loop to refine outputs. This iterative process enables light-weight TRM models to outperform larger transformer-based models on complex reasoning tasks, such as puzzles Jolicoeur-Martineau [2025]. As such, the TRM is a highly promising replacement to transformers, so we initially aimed to improve the reasoning depth of the TRM through scaling the 5 million parameter model released by Samsung, adding additional hidden states to the TRM, and extending the iterative steps the TRM could take to solve a problem. However, as we implemented our initial ideas, we found ourselves being constrained by a lack of compute.

Moreover, in October 2025, Google Deepmind released Nested Learning (NL), a new framework that develops neural models with nested memory levels, allowing these models to perform multi-level reasoning Behrouz et al. [2025]. Through adding additional hidden states, we aimed to introduce multi-level reasoning within the TRM, yet we found that this recent work explored the theoretical idea of reasoning in great depth, introducing many of the key contributions we hoped to develop. As such, we decided to shift our focus to the TITANS architecture. The TITANS architecture is a novel architecture that seeks to mimic human memory by combining a set of learned weights (mimicking persistent memory), a standard attention mechanism (mimicking working memory), and a novel neural memory module (mimicking long-term memory). The neural memory module is a dense neural network whose parameters are updated during inference; the incorporation of neural memory enables the TITANS architecture to be updated during inference, while transformers are only updated during training Behrouz et al. [2024]. As such, while modern LLMs are unable to update their parameters after training, the TITANS architecture could be used to develop more effective models that continue to learn through deployment.

1.2 Problem Statement

The TITANS architecture updates the neural memory module through a "surprise" mechanism. This update mechanism is derived primarily from gradient magnitude, with historical momentum being a secondary factor Behrouz et al. [2024]. Consequently, when high-gradient, non-task-representative samples are encountered, this exposes the model to potentially catastrophic memory updates. This hinders convergence, destabilizes descent, and introduces more noise in the loss landscape, reducing the ability of the TITANS architecture to learn through inference.

1.3 Objectives

Our primary objective in this paper is to stabilize the update mechanism of the TITANS' neural memory model by replacing the naive surprise metric with a probabilistic update. We introduce Expected Gradient Divergence Weighting (EGDW), which treats gradients and model parameters as drawn from an underlying distribution.

We implement this probabilistic update through two main forms: local surprise and global surprise. In local surprise, we use Markov's inequality to weight gradient updates inversely to the probability of the gradient magnitude occurring, filtering out unlikely noise signals. In global surprise, we use a first-order approximation of Jensen's inequality to estimate the deviation of the gradient from the expected *global* gradient, using Hessian correction terms to make up for the non-convex nature of the loss landscape.

Quantitatively, we aim to showcase the potency of EGDW through improved generalization and validation loss on a 10B token subset of the FineWebEdu dataset, an education focused dataset used in the original TITANS paper Behrouz et al. [2024].

2 Related Work and Background

The TITANS architecture is the culmination of years of prior research. As indicated previously, the TITANS architecture is built off 3 core components: persistent memory (a set of consistent weights), working memory (an attention-mechanism), and long-term memory (neural memory). The persistent memory component of the TITANS architecture is a simple set of constants, yet the working memory

and long-term memory of the TITANS architecture build off years of research into developing more effective model architectures.

Firstly, the working memory module of the TITANS architecture takes inspiration from the transformer architecture. Transformers, introduced in 2017, are based on the mechanism of self-attention. In this mechanism, each token in an input sequence directly attends to every other token, producing a weighted combination of all token representations Vaswani et al. [2023]. The working memory of the TITANS architecture utilizes self-attention when given an input sequence as self-attention allows the model to model textual relationships effectively across the sequence.

Secondly, the neural memory module of the TITANS architecture takes inspiration from prior recurrent architectures. Recent advancements in recurrent models, such as Mamba, introduce the idea of a selective state-space model (SSM). In an SSM, a hidden state is used to compress relevant information from the input sequence, and a learned gating mechanism that determines which parts of the input are relevant enough to influence the state. Unlike traditional RNNs, which indiscriminately update their hidden state at every step, Mamba selectively incorporates information based on content-dependent filters, enabling the model to retain relevant information while discarding unimportant signals Gu and Dao [2024]. Learning to Train at Test Time builds upon this idea of a dynamic hidden state by generalizing it from a simple matrix or vector to a full neural network that can be updated during inference Behrouz et al. [2024]. While these recurrent models represent impressive advancements in the field of deep learning, these recurrent models tend to struggle on very long input sequences. The hidden state in these models aims to compress all relevant information, yet the hidden state used by these models has a fixed memory capacity, so for longer input sequences, relevant information may be lost. The TITANS architecture overcomes this limitation by distributing memory across multiple components. Its working memory module, implemented through self-attention, can represent relationships across much longer contexts than a single hidden state. In parallel, its neural memory module provides a dynamic, complementary store that can retain and update information across the sequence. Together, these components enable Titans to maintain and utilize long-range information far more effectively than traditional recurrent architectures. Moreover, within the original TITANS paper, the FineWebEdu dataset is utilized. This dataset consists of educational web pages filtered from the FineWeb dataset Penedo et al. [2024]. As such, as we attempted the task of improving the neural memory fo the TITANS architecture, we opted to continue using this dataset to maintain consistency with the original work. Thus, the TITANS architecture that we intend to improve is a combination of multiple advancements (such as transformers and the Mamba architecture), and we maintain one of the original datasets used in the TITANS paper for consistency across research efforts.

3 Methodology

The models used throughout this work are Memory-as-Context Transformers based on the TITANS architecture. We replicate the configuration of the TITANS blocks used in the open source PyTorch Implementation, which is linked in the original paper Behrouz et al. [2024]. This configuration can be seen in Table 3.

For each input sequence of 1023 contiguous tokens, the model maps it to a hidden dimension of size 384 before adding axial positional encodings. Next, two forms of memory tokens are appended to the sequence: long-term and persistent. The four persistent tokens are fixed across all inputs, while the 12 long-term tokens are updated during training by the memory module.

The modified input is then passed through six identical TITANS Blocks. The structure of a single such block can be seen in Table 2. Each block splits and recombines the input representation, updates the long-term memory tokens through the memory module, performs segmented self-attention, and processes the attended features through a GeGLU feedforward block Shazeer [2020]. The output is then split and recombined once more by hyper connections to produce a richer representation.

This paper focuses exclusively on the manner in which the memory module updates the long-term memory tokens, modifying no other aspects of the architecture or configuration across runs.

3.1 Model Description

Component	Output Shape (for input $[1, 1023]$)	# Parameters
Token Embedding	$[1, 1023, 384]$	98,304
Axial Positional Embedding	$[1, 1023, 384]$	1,476,608
Stream Expansion (4 streams)	$[4, 1139, 384]$	1,536
TITANS Transformer Blocks (6 layers, identical structure)		
Block 1	$[1, 1139, 384]$	4,797,184
Block 2	$[1, 1139, 384]$	4,797,184
Block 3	$[1, 1139, 384]$	4,797,184
Block 4	$[1, 1139, 384]$	4,797,184
Block 5	$[1, 1139, 384]$	4,797,184
Block 6	$[1, 1139, 384]$	4,797,184
Stream Reduction	$[1, 1139, 384]$	–
Final RMSNorm	$[1, 1023, 384]$	384
Output Projection (Linear)	$[1, 1023, 384]$	98,304
Total		**30,651,288**

Table 1: Compact architecture summary of the 6-layer ModSurprise Memory-as-Context Transformer. Listed shapes correspond to an input sequence of length 1023 tokens. Parameter counts come from torchinfo summary.

TITANS Transformer Block	# Params
HyperConnections (input)	2,300
Memory Module (EGDW or Surprise)	1,246,979
Key/Value projections	—
Memory update rule	—
Segmented Attention	530,700
Q/K/V projections	—
Local segment masking	—
Append persistent + LT memory	—
Feedforward (GeGLU)	525,650
Hyper Connections (output)	2,300
Total per TITANS Block	**4,797,184**

Table 2: Operations and parameter counts inside a single TITANS Block (dim=384).

Hyperparameter	Value
Hidden dimension (d)	384
Number of blocks (L)	6
Segment length (S)	128
Persistent memory tokens (N_{persist})	4
Long-term memory tokens (N_{lt})	12
Vocabulary size (V)	256
Sequence length (T)	1023
Optimizer	AdamW
Learning rate	1×10^{-3}
Weight decay	1×10^{-8}
Gradient clipping	1.0
LR schedule	Cosine annealing
Training loss	Cross-entropy

Table 3: Shared configuration used for all model variants evaluated in this work.

3.2 Dataset

All models are evaluated on a 10B subset of the FineWebEdu dataset used in the original TITANS paper, a dataset specifically intended for training language models. The dataset consists of educational web pages filtered from the FineWeb dataset. During training, we select 10,000 contiguous samples of 1024 tokens each. We randomly hold out 2000 more samples to create a validation set. We tokenize each sample with the GPT-2 tokenizer, just as was done in the original TITAN paper. Each sample is processed independently, and no special pre-processing beyond the original TITAN paper is applied — this allows to attribute any change in performance to the modified surprise mechanism.

3.3 Evaluation Metric

As we are training language models, we evaluate the quality of a model through the standard cross-entropy loss on the validation set. For an input sequence of T tokens $x_{1:T}$, the model predicts a distribution over the next token at each of the T steps. The cross-entropy loss $\mathcal{L}$ is calculated as

$$-\frac{1}{T}\sum_{t=1}^{T}\log p_\theta(x_{t+1} \mid x_{\leq t}).$$

Lower cross-entropy loss is better, as the goal of EGDW is to stabilize neural memory updates and allow the TITAN architecture to generalize better, reducing overfitting to outlier gradients. We also track training loss. We consider a model that achieves a lower validation loss alongside a higher train loss to be generalizing better, and this is exactly what we observe with EGDW.

3.4 Loss Function and Optimization

As previously stated, our loss function is cross-entropy loss. All models use identical AdamW Kingma and Ba [2015] optimizers (learning rate 1×10^{-3}, weight decay 1×10^{-8}, gradient clipping to an ℓ_2 norm of at most 1.0) to reduce the number of confounding variables. Furthermore, we use a cosine-annealing learning-rate scheduler with $\eta_{\min} = 0$ and $T_{\max} = 10$. The learning rate at optimization step t is computed by

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\left(1 + \cos\left(\frac{\pi t}{T_{\max}}\right)\right).$$

3.5 Expected Gradient Divergence Weighting (EGDW)

Let us consider the mathematical intuition behind EGDW. All sources for this section come from Harchol-Balter [2024].

Let W be the model's parameter space and G be the set of all possible gradients with respect to divergence function $L : W \to \mathbb{R}$ (assuming the loss maps to a scalar value, as CE loss does). Then define a gradient function $g : W \to G$ such that $g(\theta_t)$ maps to the gradient of the given parameters $\theta_t \in W$ at training timestep t.

Let $\|g(\theta_t)\|$ represent the L2 norm of the gradient of L with respect to θ_t.

Now for a key insight: treat the gradients and model parameters as **random variables** drawn from underlying distributions G and W, respectively. We can now apply Markov's and Jensen's inequalities.

Markov's inequality tells us that unusually large gradients occur with low probability, making them statistically "surprising" events. Mathematically,

$$\Pr\big(\|g(\theta_t)\| \;>\; k\,\mathbb{E}[\|g(\theta_t)\|]\big) \;\leq\; \frac{1}{k}.$$

Jensen's inequality tells us that given a convex function (see the Error/Failure Case Analysis section for notes on how this is adapted to non-convex functions in our case), the expectation of the function exceeds the function evaluated at the expectation of its input. In this scenario, that gives

$$\mathbb{E}[\|g(\theta_t)\|] > \|g(\mathbb{E}[\theta_t])\|.$$

Thus, whenever a gradient exceeds its expected magnitude (i.e., $\|g(\theta_t)\| > \mathbb{E}[\|g(\theta_t)\|]$ for high-surprise samples), it must also exceed the gradient evaluated at the expected parameters:

$$\|g(\theta_t)\| > \|g(\mathbb{E}[\theta_t])\|.$$

Markov's inequality tells us which gradients qualify as "surprising". Jensen's inequality tells us that such "surprising" gradients should also diverge from the global reference gradient.

We now turn the above observations into a principled gradient-weighting mechanism.

Let θ_t denote the memory–MLP parameters at update step t, and let

$$g(\theta_t) = \nabla_\theta \mathcal{L}(\theta_t)$$

denote the corresponding gradient. In the original TITANs paper Behrouz et al. [2024], the "surprise" update S_t is computed as

$$S_t = \eta_{t-1} S_{t-1} - g(\theta_t),$$

where η_{t-1} is a tunable parameter.

In EGDW, we replace this with a probabilistic formula that weights gradients proportionally to their distance from both their local gradient history and a global reference gradient. To do this, we maintain two exponential moving averages with smoothing coefficient $\alpha \in (0, 1)$: a running estimate of the expected gradient magnitude and a running estimate of the parameters. These are defined as

$$E[\|g(\theta_t)\|]_t = \frac{\alpha}{n} \sum g(\theta_t) + (1 - \alpha)\, E[\|g(\theta_t)\|]_{t-1},$$

$$E[\theta]_t = \frac{\alpha}{n}\, \theta_t + (1 - \alpha)\, E[\theta]_{t-1}.$$

The global reference gradient is then given by

$$g\big(E[\theta]_t\big) = \nabla_\theta \mathcal{L}(E[\theta]_t)\,.$$

We compute the surprise update as the product of two factors: a local surprise and a global surprise. The local surprise measures how much the current gradient deviates from its local gradient history, while the global surprise measures its deviation from the global reference gradient.

$$L_t \;=\; \frac{\|g(\theta_t)\|}{E[\|g(\theta_t)\|]_t}, \qquad G_t \;=\; \frac{\|g(\theta_t)\|}{\|g(E[\theta]_t)\|}.$$

Our EGDW surprise update scales the gradient by the product of these two terms:

$$S_t \;=\; (L_t\, G_t) \cdot g(\theta_t).$$

This means that gradients are only upweighted when they are 'surprising' both locally and globally; otherwise, they are proportionally downweighted.

4 Baseline and Extensions

4.1 Baseline Selection and Evaluation

As our entire work focuses on improving the memory update mechanism in a TITAN model, it only makes sense to use the default TITAN model as a baseline. We ran the default TITAN models 4 times using the open source Python implementation Behrouz et al. [2024]. We then evaluated a set of modified memory update rules. We considered the default update, multiplicative EGDW, additive EGDW, and multiplicative EGDW with Hessian correction terms. In total, we conducted 27 modified runs.

4.2 Implemented Extensions

We discuss and analyze multiple implemented extensions of the Titan architecture in section 5.

4.3 Baseline Reproduction Evidence

Figure 1 contains clear evidence of multiple baseline reproduction.

5 Results and Analysis

5.1 Results

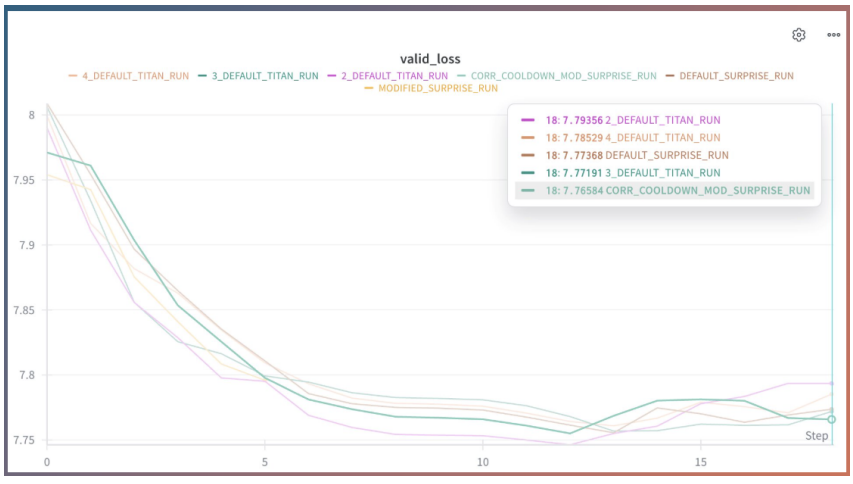


Figure 1: Validation cross-entropy for all baselines and EGDW-TITANS. Lower is better.



Figure 2: Training cross-entropy for all baselines and EGDW-TITANS. EGDW maintains higher training loss but consistently achieves lower validation loss, indicating better generalization.

When analyzing the aforementioned graphs, observe that our run exhibits more stable descent throughout. The descent graph with respect to validation loss is far smoother, and the only run that appears to do better, 2_DEFAULT_TITAN_RUN, cannot retain the gains it makes and eventually diverges, finishing as the worst model of the bunch.

By the end, our modified surprise model remains the best performer in terms of validation loss despite having the second-highest training loss, indicating that the model's learnings are more tied towards task-performance rather than memorizing the training dataset, which correlates to convergence towards wider minima which as been shown in empirical work to lead to stronger real-world task performance and generalization Wen et al. [2018]. Namely, this means EGDW surprise appears to enhance generalization, as we initially hypothesized.

5.2 Error / Failure Case Analysis

The usage of Jensen's inequality Harchol-Balter [2024] to relate the expected gradient magnitude to the gradient at the expected parameters is only completely valid when $g(\theta_t)$ is a convex function. While training deep neural networks, however, it is common to obtain a non-convex loss landscape. In this case, one must compensate for the difference using the Hessian principal eigenvector projections, adding second-order projection terms along the principal eigenvectors in accordance with the second-order Taylor Series (higher-order terms can also be added for more accuracy) Harchol-Balter [2024]. While this would in theory yield a tighter bound and more accurate estimations of global surprise, in practice this turned out to be extremely computationally expensive, especially upon the calculation of these Hessian correction terms for each parameter group. The memory of the A100 GPU we used to train these models was exhausted in just one optimization step. Given the prohibitively expensive cost for relatively little tradeoff, we decided to stick with first-order approximations of Jensen's inequality for estimating global surprise.

5.3 Sensitivity and Ablation Analysis

As part of our ablation study, we first analyzed a version of EGDW that added the local and global surprise terms rather than multiplying them [Figure 3]. Empirically, the additive interaction performed significantly worse than even the baseline Titan runs. Training became unstable in the first few epochs, and we terminated the runs early due to poor performance. We hypothesize that this additive form allowed just one of local or global surprise to dominate the gradient update, counteracting the entire point of EGDW: a large gradient magnitude should greatly affect the neural memory module only if it is both locally and globally surprising. The multiplicative formula naturally enforces this condition, preventing domination by just one term and leading to stabler training.

Secondly, we tested a version of the Titan that consists of **special cooldown steps** that apply regular surprise before switching to EGDW. In our empirical testing, we found that at the initial few epochs,

the updates of our modified surprise TITANS architecture could be highly unstable. The model would recover a few epochs later and then again overtake the regular surprise models, and so we simply decided to see what would happen if EGDW surprise was turned off for the first **2** epochs (which is the hyperparameter we ended up running for all of our runs). After doing so, training stabilized for our modified surprise variant.

Mathematically, the reason for this is clear - at the beginning of training, our running average estimates of the expected value of the gradient and the running average of the parameters (both of which serve as estimates of the expected gradient and expected parameter values) are highly inaccurate since fewer samples have been observed. Consequently, the divergence factor from the expected gradient and the global gradient of the expected parameter values has the potential to be wildly inaccurate, and hence, it is more mathematically sound for one to update these values after a few epochs have been run and the expected value estimations have had more iterations to stabilize.

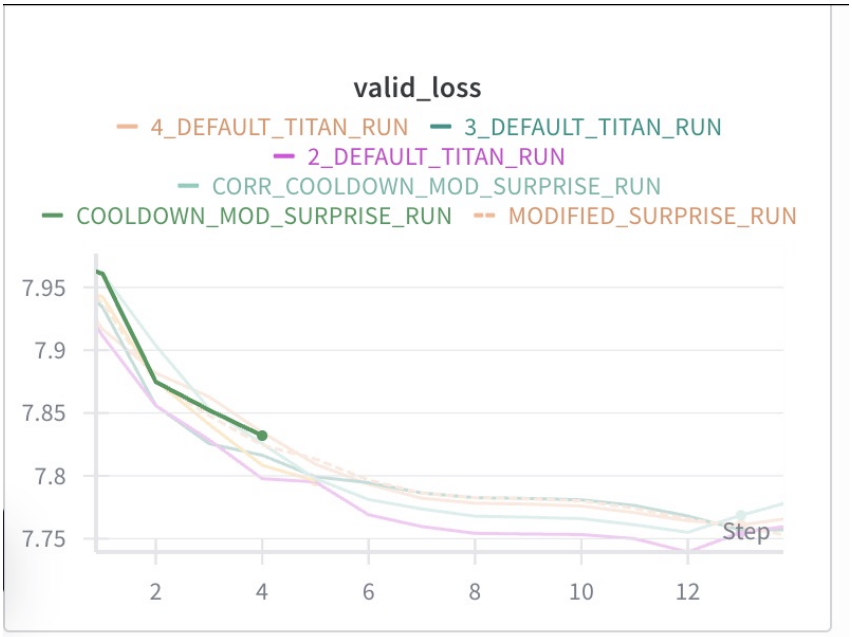


Figure 3: Training and validation loss for additive version of EGDW

6 Future Discussion

Our experiments demonstrate that such probabilistic gradient weighting can improve the stability and generalization of the TITAN's neural memory updates in this setting. There are many ways in which this work can be extended, four of which are as follows:

1. Our original motivation involved hierarchical reasoning models such as the TRM and HRM. One interesting idea is to use EGDW memory updates in these hierarchical models, as local and global "surprise" calculations naturally align with then lower level reasoning modules and higher level controller modules found in the TRM.
2. Due to compute constraints, we evaluated EGDW-TITANS only on a relatively small model for a relatively small number of epochs (19), and a 10B-token subset of FineWebEdu. Running the same experiments for a much longer horizon (100+ epochs) and for much larger models (360M+ parameters) with larger context windows could allow us to truly test the effect of EGDW for extremely long context tasks.
3. We noticed that delaying the application of EGDW for a few epochs allows a model's running averages to stabilize, preventing wild oscillations in neural memory updates in early epochs. One might experiment with modifying the transition from surprise-based memory updates to EGDW by gradually annealing away the surprise while increasing the weightage

of EGDW over time. A natural direction for future work is to study this warmup period systematically, determining the optimal warmup time for models of different architectures or scales.

4. Finally, EGDW is not necessarily specific to language modeling. Applying this framework to other machine learning fields (e.g. reinforcement learning, vision models) could test its potential as a technique not just for language models but for neural networks in general.

7 Conclusion

In this work, we proposed *Expected Gradient Divergence Weighting* (EGDW) as a probabilistic alternative to the hyperparameter-based memory update used in the TITANS architecture. Rather than relying directly on raw gradient magnitude, EGDW treats gradients and memory parameters as random variables and weights updates by how surprising they are both locally (relative to recent gradient history) and globally (relative to a running estimate of the global gradient).

On a 10B token subset of FineWebEdu, EGDW-Titans consistently outperformed baseline Titans in terms of validation cross entropy loss while maintaining a higher training loss, suggesting better generalization. While we tested other variants of EGDW, such as adding the local and global surprise or employing Hessian corrective terms, we either saw a decrease in performance or hit compute limits, highlighting the relative value of this simple multiplicative rule against its additional compute cost.

Overall, our results provide evidence that probabilistic memory updates have the capacity to improve the long-context ability of language models. While our experiments are limited to a single architecture, dataset, and model size, they suggest that modifying memory updates based on mathematical bounds rather than tunable hyperparameters has the potential to be a fruitful direction of further efforts.

8 Administrative Details

8.1 Bibliography

References

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, 2024. URL https://arxiv.org/abs/2501.00663.

Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. Nested learning: The illusion of deep learning architectures. In *Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025. URL https://openreview.net/pdf?id=nbMeRvNb7A. Poster presentation, CC BY 4.0.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=tEYskw1VY2.

Mor Harchol-Balter. *Introduction to Probability for Computing*. Cambridge University Press, 2024.

Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025. URL https://arxiv.org/abs/2510.04871.

Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2015. URL https://arxiv.org/abs/1412.6980.

Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale, 2024. URL https://arxiv.org/abs/2406.17557.

Noam Shazeer. Glu variants improve transformer, 2020. URL https://arxiv.org/abs/2002.05202.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL https://arxiv.org/abs/1706.03762.

Wei Wen, Yandan Wang, Feng Yan, Cong Xu, Chunpeng Wu, Yiran Chen, and Hai Li. Smoothout: Smoothing out sharp minima to improve generalization in deep learning, 2018. URL https://arxiv.org/abs/1805.07898.

8.2 Team Contributions

Initial research, ideation, mathematical justification, implementation of the project, and report writing was split evenly between group members.

8.3 Github

All code can be found at https://github.com/pranavr11/IDL_FINAL_PROJ.